Module 4 Hands-On Lab

Google OAuth Login — Full Guide

Everything about the "Sign in with Google" part of this lab: what it actually does, how to set it up, how it connects to the code, and how to fix it when it breaks.

---

1. What OAuth actually is

Your app never sees a user's Google password, and it never should. Instead, the login flow works like this:

1. User clicks **"Sign in with Google"** on your app's login page.
2. The browser is redirected *away* from your app, to Google's own login page.
3. The user enters their password there — on Google's site, never on yours.
4. Google confirms the user's identity, then redirects the browser **back** to a specific URL your app registered in advance, attaching a short-lived authorization code.
5. Your app exchanges that code with Google for confirmed identity info (email, name), creates a session, and logs the user in.

The whole point: your app gets to say "this is genuinely jane@gmail.com" without ever handling her password, storing it, or being responsible for it.

2. What each tool in the stack does

- **Google** — verifies identity only. Confirms who the user is, then it's done — it doesn't keep them logged into your app.
- **NextAuth** — the glue. Speaks Google's OAuth protocol, creates a session after Google confirms identity, issues a cookie, and checks that cookie on every future request.
- **Prisma** (via the NextAuth adapter) — stores the session as a real row in your Postgres database, rather than only inside the cookie.

---

3. Setting up credentials in Google Cloud Console

Step 1 — Create a project

Go to **console.cloud.google.com** and sign in with any Google account. Click the project dropdown near the top-left → **New Project** → give it a name (e.g. `orders-lab-demo`) → Create. Make sure it's selected in the dropdown afterward.

Step 2 — Configure the OAuth consent screen

Go to **APIs & Services → OAuth consent screen**.

- User Type: **External** → Create
- App name: anything descriptive, e.g. `Orders Lab Demo`
- User support email and developer contact email: your own email
- Click through **Save and Continue** on the Scopes and Test Users screens — defaults are fine for local development

Step 3 — Create the OAuth client ID

Go to **APIs & Services → Credentials → + Create Credentials → OAuth client ID**.

- Application type: **Web application**
- Name: anything, e.g. `orders-lab-web`
- Under **Authorized redirect URIs**, click **+ Add URI** and enter exactly:

```
http://localhost:3000/api/auth/callback/google
```

> **Note:** this must match **character for character** — no trailing slash, `http` not `https`, port `3000` exactly. Google rejects the login if it doesn't match precisely; this isn't a formality, it's what stops a malicious site from registering itself to receive someone else's login.

Click **Create**. A popup shows your **Client ID** and **Client Secret** — keep this open, you'll need both in the next section.

---

4. Wiring credentials into the project

Open `.env` in your project folder and fill in:

```
GOOGLE_CLIENT_ID="paste-your-client-id-here"
GOOGLE_CLIENT_SECRET="paste-your-client-secret-here"
```

Two other values in `.env` that OAuth login depends on:

```
NEXTAUTH_URL="http://localhost:3000"
AUTH_SECRET="<generate with: npx auth secret>"
```

> **Note:** if `npx auth secret` outputs a line called `BETTER_AUTH_SECRET` instead of `AUTH_SECRET`, rename it — this project reads that exact variable name, and login will fail on startup with a `MissingSecret` error otherwise.

Restart the dev server after any `.env` change — environment variables are only read once, at server start:

```
npm run dev
```

---

5. Where this shows up in the code

You don't need to write any OAuth logic by hand — the `Google` provider and the `PrismaAdapter` in `auth.ts` handle the entire protocol:

```
// auth.ts
export const { handlers, auth, signIn, signOut } = NextAuth({
  adapter: PrismaAdapter(prisma),   // store sessions in Postgres
  providers: [
    Google({
      clientId: process.env.GOOGLE_CLIENT_ID,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET,
    }),
  ],
  session: { strategy: "database" },
```

```
  pages: { signIn: "/login" },
});
```

The callback URL you registered in Google Console is automatically handled by:

```
app/api/auth/[...nextauth]/route.ts
```

You never write logic for that file's contents — it just re-exports `handlers` from `auth.ts`, and NextAuth's internals do the rest.

---

6. Troubleshooting

Error	Cause	Fix
`Error 401: invalid_client`	.env missing, or `GOOGLE_CLIENT_ID`/`SECRET` are placeholders or don't match Google Cloud Console	Confirm `.env` exists in this exact project folder with real values; restart `npm run dev` after editing
`redirect_uri_mismatch`	Redirect URI registered in Google Console doesn't exactly match what the app sends	Must be exactly `http://localhost:3000/api/auth/callback/google` — no trailing slash, no `https`
`MissingSecret` / `Please define a secret`	`AUTH_SECRET` missing from `.env`, or named `BETTER_AUTH_SECRET` instead	Run `npx auth secret`; if it outputs `BETTER_AUTH_SECRET`, rename that line to `AUTH_SECRET`
"Google hasn't verified this app" warning	Expected — the OAuth consent screen is in testing/unpublished mode, normal for local dev	Click **Advanced → Go to (app name) (unsafe)** to proceed
Logged in but orders list is empty	Seeded demo data belongs to a different email than the one you logged in with	Edit `prisma/seed.ts` with your real email, re-run `npm run seed`
Works in one project folder, breaks in a copy/clone	`.env` is intentionally excluded from git/zip — a fresh copy has none	`copy .env.example .env` and refill every value in the new folder

---

7. Security notes

- `GOOGLE_CLIENT_SECRET` and `AUTH_SECRET` are real secrets. Never commit `.env` to git — it's already excluded via `.gitignore` in this project, but always double-check with `git status` before your first commit.
- The exact-match requirement on the redirect URI exists specifically to prevent a malicious site from registering a lookalike URL and intercepting someone else's login handoff.
- If you ever suspect `AUTH_SECRET` has leaked, generate a new one and restart the server — this immediately invalidates every existing session, forcing everyone to log in again.